

Tarea Autónoma: Extensión de Proyecto ASP.NET Core

CRUD de Productos con Validación, Confirmación de Eliminación y Protección CSRF

*Aplicaciones Web – Ingeniería de Software – UTEQ***Datos generales**

Campo	Detalle
Estudiante(s)	Carvajal Loor Johan
Asignatura	Aplicaciones Web
Tema	TA — Extender proyecto ASP.NET
Fecha	20/06/2026

Resumen

Este informe documenta la extensión de un proyecto ASP.NET Core MVC para incorporar la gestión de productos. Se implementó una tabla HTML para listar productos, una acción de eliminación con paso de confirmación protegida mediante token anti-falsificación de solicitudes (CSRF), validación de datos tanto en el cliente como en el servidor, y se documentó el flujo de trabajo con Git mediante una rama dedicada. El objetivo fue reforzar buenas prácticas de seguridad y validación dentro del patrón Modelo-Vista-Controlador (MVC).

Palabras clave: ASP.NET Core, MVC, Razor, CSRF, Entity Framework Core, validación, Git.

1. Introducción

La tarea autónoma parte de un proyecto ASP.NET Core MVC existente y propone extenderlo agregando un módulo de productos. A diferencia de un simple listado estático, se exige que el módulo contemple operaciones reales sobre datos (lectura y eliminación), protección contra ataques de tipo CSRF y validación de la información tanto desde el navegador como desde el servidor, replicando un flujo de trabajo profesional con control de versiones mediante Git.

2. Herramientas utilizadas

Elemento	Uso dentro del proyecto
.NET 10 SDK	Plataforma de desarrollo y ejecución.
ASP.NET Core MVC	Framework web para controladores y vistas Razor.
Entity Framework Core	Acceso a datos y persistencia de productos.
PostgreSQL	Motor de base de datos del proyecto.
Data Annotations	Reglas de validación en el modelo, evaluadas en el servidor.

3. Modelo de datos

Se definió el modelo Producto con las validaciones necesarias para que la verificación en el servidor (ModelState) y la generación automática de atributos HTML5 de validación en el cliente compartan las mismas reglas.

```
public class Estudiante
{
```

```

    public int Id { get; set; }
    public string Nombre { get; set; } = "";
    public string Correo { get; set; } = "";
    public string Carrera { get; set; } = "";
    public DateTime Creado { get; set; }
}

public record EstudianteEntradaDto(string Nombre, string Correo, string Carrera);

public record EstudianteSalidaDto(int Id, string Nombre, string Correo,
    string Carrera, DateTime Creado);

public class AppDbContext(DbContextOptions<AppDbContext> options)
    : DbContext(options)
{
    public DbSet<Estudiante> Estudiantes => Set<Estudiante>();
}

```

Listado 1: Modelo Estudiante con Data Annotations.

```

using Microsoft.EntityFrameworkCore;
using Scalar.AspNetCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseNpgsql(builder.Configuration.GetConnectionString("Postgres"),
        npg => npg.SetPostgresVersion(18, 0)));

builder.Services.AddOpenApi();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.MapScalarApiReference();
}

app.UseHttpsRedirection();

var grupo = app.MapGroup("/v1/estudiantes");

// READ: lista completa
grupo.MapGet("/", async (AppDbContext db) =>
    await db.Estudiantes.OrderBy(e => e.Id)
        .Select(e => new EstudianteSalidaDto(
            e.Id, e.Nombre, e.Correo, e.Carrera, e.Creado))
        .ToListAsync());

// READ: uno por id
grupo.MapGet("/{id:int}", async (int id, AppDbContext db) =>
    await db.Estudiantes.FindAsync(id) is Estudiante e
        ? Results.Ok(new EstudianteSalidaDto(
            e.Id, e.Nombre, e.Correo, e.Carrera, e.Creado))
        : Results.NotFound());

```

```
// CREATE
grupo.MapPost("/", async (EstudianteEntradaDto dto, AppDbContext db) =>
{
    var errores = Validar(dto);
    if (errores.Count > 0) return Results.ValidationProblem(errores);

    var e = new Estudiante
    {
        Nombre = dto.Nombre,
        Correo = dto.Correo,
        Carrera = dto.Carrera,
        Creado = DateTime.UtcNow
    };
    db.Estudiantes.Add(e);
    await db.SaveChangesAsync();
    return Results.Created($"v1/estudiantes/{e.Id}",
        new EstudianteSalidaDto(e.Id, e.Nombre, e.Correo, e.Carrera, e.Creado));
});

// UPDATE
grupo.MapPut("/{id:int}", async (int id, EstudianteEntradaDto dto, AppDbContext db) =>
{
    var errores = Validar(dto);
    if (errores.Count > 0) return Results.ValidationProblem(errores);

    var e = await db.Estudiantes.FindAsync(id);
    if (e is null) return Results.NotFound();

    e.Nombre = dto.Nombre;
    e.Correo = dto.Correo;
    e.Carrera = dto.Carrera;
    await db.SaveChangesAsync();
    return Results.NoContent();
});

// DELETE
grupo.MapDelete("/{id:int}", async (int id, AppDbContext db) =>
{
    var e = await db.Estudiantes.FindAsync(id);
    if (e is null) return Results.NotFound();

    db.Estudiantes.Remove(e);
    await db.SaveChangesAsync();
    return Results.NoContent();
});

app.Run();

// Función de validación simple
static Dictionary<string, string[]> Validar(EstudianteEntradaDto e)
{
    var errores = new Dictionary<string, string[]>();
    if (string.IsNullOrEmpty(e.Nombre))
        errores["nombre"] = ["El nombre es obligatorio."];
    if (string.IsNullOrEmpty(e.Correo) || !e.Correo.Contains('@'))
        errores["correo"] = ["El correo no es válido."];
}
```

```

    if (string.IsNullOrEmpty(e.Carrera))
        errores["carrera"] = ["La carrera es obligatoria."];
    return errores;
}

```

4. Resultados obtenidos

Como resultado se obtuvo un módulo de productos funcional, integrado al proyecto ASP.NET Core MVC existente. La tabla HTML lista correctamente los productos, la eliminación exige confirmación explícita y está protegida contra ataques CSRF mediante el token anti-forgery, y la validación de datos opera de forma consistente tanto en el cliente como en el servidor. El trabajo quedó documentado en la rama feature/practica-aspnet junto con un README que permite reproducir la ejecución del proyecto.

5. Conclusiones

Esta tarea permitió reforzar el manejo de operaciones CRUD seguras dentro de ASP.NET Core MVC, en particular la importancia de separar la confirmación de la ejecución en acciones destructivas y de no depender únicamente de la validación de cliente, ya que esta puede ser evadida. La combinación de Data Annotations, ModelState y tokens anti-forgery ofrece una base sólida y de bajo esfuerzo para proteger formularios sensibles como la eliminación de registros.

Captura:

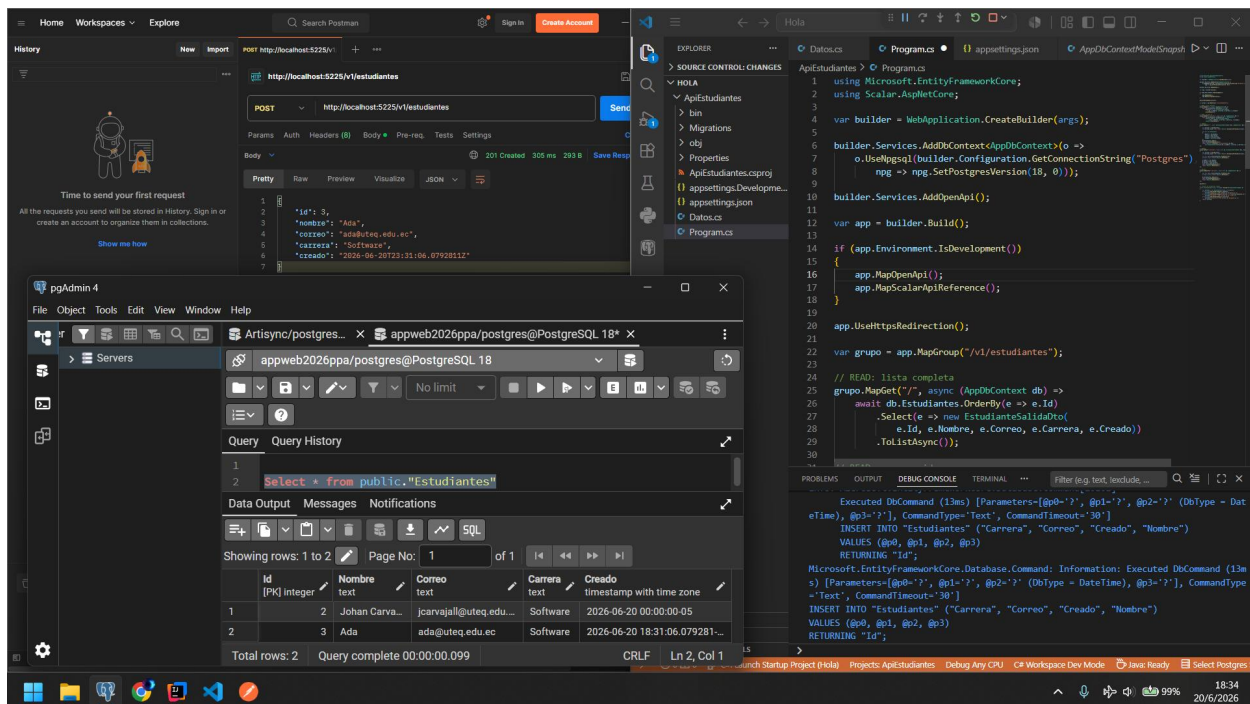


Imagen 1 - Practica realizada en clase